

Dokumentasi Jobsheet 1 – SIMPUS-Mini

Dokumentasi ini dibuat khusus untuk mahasiswa yang baru belajar HTML/CSS. Setiap file kode di jobsheet-01 dijelaskan baris per baris: tag apa yang dipakai, kenapa dipakai, dan apa gunanya di dunia nyata.

Daftar Isi

1. [Konsep Dasar yang Dipakai di Jobsheet Ini](#)
2. [Penjelasan `index.html` \(Halaman Beranda\)](#)
3. [Penjelasan `buku/list.html` \(Daftar Buku\)](#)
4. [Penjelasan `buku/tambah.html` \(Form Tambah Buku\)](#)
5. [Penjelasan `anggota/list.html` \(Daftar Anggota\)](#)
6. [Penjelasan `anggota/tambah.html` \(Form Tambah Anggota\)](#)
7. [Rangkuman & Latihan Lanjutan](#)

Tentang Program Ini

Jobsheet 1 membangun kerangka awal aplikasi **SIMPUS-Mini**, yaitu sistem perpustakaan mini yang bisa mengelola data **buku** dan **anggota**. Di tahap ini aplikasi **belum bisa menyimpan data sungguhan** — semua data di tabel masih data dummy (contoh) yang ditulis manual di HTML, dan form belum diproses ke mana pun. Fokus jobsheet ini murni pada:

- Struktur halaman web memakai **HTML5 semantic** (`header`, `nav`, `main`, `section`, `article`, `footer`), bukan `div` polos di semua tempat.
- Penamaan atribut `id` dan `name` pada form, yang nantinya akan dipakai lagi ketika belajar CSS (jobsheet 2) dan JavaScript/backend (jobsheet berikutnya).

Struktur Folder

```
jobsheet-01/
├── index.html           # Halaman beranda
├── buku/
│   ├── list.html       # Tabel daftar buku (statis)
│   └── tambah.html     # Form tambah buku (belum diproses)
├── anggota/
│   ├── list.html       # Tabel daftar anggota (statis)
│   └── tambah.html     # Form tambah anggota (belum diproses)
```

— README.md # Ringkasan singkat jobsheet (dari dosen)
— dokumentasi/ # Folder dokumentasi ini

Silakan baca urut dari nomor 1 supaya konsepnya nyambung, atau langsung loncat ke file yang ingin dipahami.

1. Konsep Dasar yang Dipakai di Jobsheet Ini

Sebelum masuk ke penjelasan tiap file, kenali dulu istilah-istilah yang akan terus muncul di semua file HTML jobsheet ini.

1.1 Apa itu "tag" dan "elemen"?

HTML ditulis memakai **tag** yang biasanya berpasangan: tag pembuka `<namatag>` dan tag penutup `</namatag>`. Isi di antara keduanya disebut **elemen**.

```
<h1>SIMPUS-Mini</h1>
```

`<h1>` adalah tag pembuka, `</h1>` tag penutup, dan seluruh potongan kode di atas adalah elemen `h1` (judul level 1).

1.2 Struktur Wajib Setiap Halaman HTML

Semua file di jobsheet ini diawali dengan pola yang sama:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>SIMPUS-Mini | Beranda</title>
</head>
<body>
  ...
</body>
</html>
```

Bagian	Fungsi
<code><!DOCTYPE html></code>	Memberi tahu browser bahwa dokumen ini HTML5. Wajib ada di baris paling atas.

Bagian	Fungsi
<code><html lang="id"></code>	Elemen pembungkus seluruh halaman. <code>lang="id"</code> memberi tahu browser (dan screen reader) bahwa isi halaman berbahasa Indonesia.
<code><head></code>	Berisi informasi <i>tentang</i> halaman yang tidak tampil di layar: judul tab, encoding karakter, link CSS, dsb.
<code><meta charset="UTF-8"></code>	Mengatur pengkodean karakter jadi UTF-8, supaya karakter seperti é, — , atau huruf non-Latin tampil benar, tidak jadi simbol aneh.
<code><title></code>	Teks yang muncul di tab browser . Perhatikan tiap halaman punya title berbeda (<code>SIMPUS-Mini Beranda</code> , <code>SIMPUS-Mini Daftar Buku</code> , dst.) supaya pengguna tahu sedang di halaman mana.
<code><body></code>	Berisi semua konten yang tampil di layar: teks, gambar, tabel, form, dll.

1.3 Tag Semantic HTML5

"Semantic" artinya nama tag menjelaskan **makna/peran** kontennya, bukan sekadar kotak kosong seperti `<div>`. Ini yang dipakai di jobsheet ini:

- `<header>` — bagian kepala halaman, biasanya berisi judul situs dan navigasi. Muncul di **semua** halaman jobsheet ini dengan isi yang mirip (judul `SIMPUS-Mini` + menu `nav`).
- `<nav>` — kelompok tautan navigasi (menu). Di dalamnya dipakai `<ul>` / `<li>` / `<a>` biasa, tapi dibungkus `<nav>` supaya browser/screen reader tahu ini adalah "menu", bukan daftar isi biasa.
- `<main>` — konten **utama** halaman. Hanya boleh ada **satu** `<main>` per halaman.
- `<section>` — mengelompokkan konten yang temanya sama, biasanya diawali sebuah heading (`<h2>`, dst.).
- `<article>` — potongan konten yang berdiri sendiri/bisa dipisah dari konteks sekitarnya. Di `index.html`, tiap kartu statistik (Total Buku, Total Anggota, Sedang Dipinjam) dibungkus `<article>` karena masing- masing adalah info yang berdiri sendiri.
- `<footer>` — bagian kaki halaman, biasanya berisi copyright/info tambahan. Sama di semua halaman: © 2026 SIMPUS-Mini – Jobsheet 1.

Kenapa tidak pakai `<div>` saja untuk semuanya? Karena `<div>` tidak membawa makna apa pun — browser, mesin pencari, dan alat bantu difabel (screen reader) tidak tahu apakah sebuah `<div>` itu menu, konten utama, atau footer. Dengan tag semantic, kode lebih mudah dibaca **dan** lebih ramah aksesibilitas & SEO.

1.4 Kenapa Belum Ada CSS?

Sesuai catatan di [README.md](#) jobsheet ini, tahap ini **sengaja belum diberi CSS/JavaScript**. Tujuannya supaya kamu fokus dulu pada struktur (kerangka) halaman. Karena belum ada CSS, tampilan di browser akan terlihat polos (font default, tanpa warna/tata letak rapi) — itu **normal**, bukan kesalahan. Styling akan dipelajari di jobsheet berikutnya.

1.5 Navigasi Antar Halaman (`<a href="...">`)

Perhatikan menu `<nav>` di setiap file memakai path relatif berbeda-beda tergantung lokasi file itu sendiri, misalnya:

- Dari `index.html` (di folder root) ke daftar buku: `href="buku/list.html"` (turun ke folder buku).
- Dari `buku/list.html` kembali ke beranda: `href="../index.html"` (`../` berarti naik satu folder ke atas).
- Dari `buku/list.html` ke `buku/tambah.html`: cukup `href="tambah.html"` karena masih di folder yang sama.

Ini penting dipahami karena kesalahan path relatif adalah salah satu penyebab paling umum tautan "tidak jalan" saat belajar HTML.

Lanjut ke: [Penjelasan index.html](#)

2. Penjelasan `index.html` (Halaman Beranda)

File ini adalah **halaman pertama** yang dibuka ketika aplikasi dijalankan (karena namanya `index.html`, nama baku yang otomatis dicari browser/server sebagai halaman utama sebuah folder).

2.1 Kode Lengkap

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>SIMPUS-Mini | Beranda</title>
</head>
<body>
  <header>
    <h1>SIMPUS-Mini</h1>
    <nav>
```

```

<ul>
  <li><a href="index.html">Beranda</a></li>
  <li><a href="buku/list.html">Daftar Buku</a></li>
  <li><a href="buku/tambah.html">Tambah Buku</a></li>
  <li><a href="anggota/list.html">Daftar Anggota</a></li>
</ul>
</nav>
</header>

<main>
  <section>
    <h2>Selamat Datang di Sistem Perpustakaan Mini</h2>
    <p>Aplikasi sederhana untuk mengelola data buku dan anggota perpustakaan.</p>
  </section>

  <section>
    <h2>Ringkasan</h2>
    <article>
      <h3>Total Buku</h3>
      <p>12</p>
    </article>
    <article>
      <h3>Total Anggota</h3>
      <p>8</p>
    </article>
    <article>
      <h3>Sedang Dipinjam</h3>
      <p>3</p>
    </article>
  </section>
</main>

<footer>
  <p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 1</p>
</footer>
</body>
</html>

```

2.2 Penjelasan Bagian per Bagian

<header> — Kepala Halaman

```

<header>
  <h1>SIMPUS-Mini</h1>
  <nav>...</nav>
</header>

```

- `<h1>SIMPUS-Mini</h1>` adalah judul utama aplikasi. Setiap halaman HTML sebaiknya hanya punya **satu** `<h1>` — ini heading level tertinggi.
- Di dalam `<header>` ada `<nav>` yang berisi menu navigasi ke 4 halaman: Beranda, Daftar Buku, Tambah Buku, dan Daftar Anggota. Perhatikan tidak ada tautan "Tambah Anggota" di menu ini — itu wajar, karena tugas mandiri (lihat [dokumentasi anggota/tambah.html](#)) memang meminta kamu mempraktikkan sendiri konsistensi menu ini.

`<nav>` — Menu Navigasi

```
<nav>
  <ul>
    <li><a href="index.html">Beranda</a></li>
    <li><a href="buku/list.html">Daftar Buku</a></li>
    <li><a href="buku/tambah.html">Tambah Buku</a></li>
    <li><a href="anggota/list.html">Daftar Anggota</a></li>
  </ul>
</nav>
```

- `<ul>` (unordered list / daftar tak berurutan) membungkus daftar menu.
- Tiap `<li>` (list item) adalah satu item menu, berisi satu tautan `<a>`.
- `href` pada setiap `<a>` menentukan halaman tujuan. Karena `index.html` berada di folder **root** jobsheet-01, path ke folder `buku/` dan `anggota/` ditulis langsung tanpa `../` (lihat [konsep dasar §1.5](#)).

`<main>` — Konten Utama

Berisi dua `<section>`:

Section 1 — Sambutan

```
<section>
  <h2>Selamat Datang di Sistem Perpustakaan Mini</h2>
  <p>Aplikasi sederhana untuk mengelola data buku dan anggota perpustakaan.</p>
</section>
```

Sekadar teks sambutan: satu `<h2>` (judul sub-bagian, satu level di bawah `<h1>`) dan satu paragraf `<p>`.

Section 2 — Ringkasan Statistik

```
<section>
  <h2>Ringkasan</h2>
  <article>
    <h3>Total Buku</h3>
```

```

        <p>12</p>
    </article>
    <article>
        <h3>Total Anggota</h3>
        <p>8</p>
    </article>
    <article>
        <h3>Sedang Dipinjam</h3>
        <p>3</p>
    </article>
</section>

```

- Setiap `<article>` di sini mewakili satu "kartu" statistik yang berdiri sendiri: judul kecil (`<h3>`, satu level di bawah `<h2>`) + angka (`<p>`).
- Angka 12, 8, 3 di sini **hanya data contoh (dummy)**, diketik manual. Belum ada logika yang menghitung dari data asli — itu akan dipelajari saat masuk ke pemrograman sisi server/JavaScript.
- Kenapa dipilih `<article>` dan bukan `<div>`? Karena tiap kartu bisa "berdiri sendiri" secara makna — kalau dipindah ke halaman lain pun, informasinya tetap utuh dan masuk akal (misalnya "Total Buku: 12").

`<footer>` — Kaki Halaman

```

<footer>
    <p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 1</p>
</footer>

```

- `©` adalah **HTML entity** (kode karakter khusus) untuk simbol ©.
- `—` adalah entity untuk tanda pisah panjang – (em dash). Entity dipakai karena beberapa karakter simbol tidak selalu bisa/aman diketik langsung di HTML, jadi ditulis dalam bentuk kode ini.

2.3 Kesimpulan

`index.html` menunjukkan pola dasar yang **diulang** di semua halaman lain di jobsheet ini: header (judul + nav) → main (isi khusus halaman) → footer (copyright). Setelah paham file ini, kamu akan lebih mudah memahami 4 file HTML lainnya karena kerangkanya sama, hanya isi `<main>`-nya yang berbeda.

Lanjut ke: [Penjelasan buku/list.html](#)

3. Penjelasan buku/list.html (Daftar Buku)

File ini menampilkan **tabel** berisi daftar buku perpustakaan (data contoh/statis, 5 baris).

3.1 Kerangka Halaman

Bagian `<header>` dan `<footer>` polanya **sama persis** dengan `index.html` (lihat [dokumentasi index.html](#)), hanya saja path pada `href` berubah karena file ini berada di dalam folder `buku/`, bukan di root:

```
<li><a href="../index.html">Beranda</a></li>
<li><a href="list.html">Daftar Buku</a></li>
<li><a href="tambah.html">Tambah Buku</a></li>
<li><a href="../anggota/list.html">Daftar Anggota</a></li>
```

- `../index.html` → naik satu folder ke root, baru buka `index.html`.
- `list.html` dan `tambah.html` → tanpa awalan, karena keduanya ada di folder `buku/` yang sama.
- `../anggota/list.html` → naik satu folder ke root, lalu turun ke folder `anggota/`.

Bagian yang benar-benar baru di file ini ada di dalam `<main>`, yaitu **tabel HTML**.

3.2 Anatomi Tabel HTML

```
<table>
  <thead>
    <tr>
      <th>Judul</th>
      <th>Pengarang</th>
      <th>Tahun</th>
      <th>Stok</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Laskar Pelangi</td>
      <td>Andrea Hirata</td>
      <td>2005</td>
      <td>4</td>
      <td>
        <button type="button">Edit</button>
        <button type="button">Hapus</button>
      </td>
    </tr>
    <!-- ... 4 baris lain dengan pola yang sama ... -->
```

```
</tbody>
</table>
```

Tag	Kepanjangan	Fungsi
<code><table></code>	table	Pembungkus seluruh tabel.
<code><thead></code>	table head	Bagian kepala tabel — berisi nama-nama kolom.
<code><tbody></code>	table body	Bagian isi tabel — berisi baris-baris data sesungguhnya.
<code><tr></code>	table row	Satu baris tabel (horizontal).
<code><th></code>	table header cell	Satu sel judul kolom (teks otomatis tebal & rata tengah oleh browser).
<code><td></code>	table data cell	Satu sel data biasa di dalam baris.

Cara membaca struktur: `<table>` berisi baris-baris (`<tr>`), dan setiap baris berisi sel-sel (`<th>` untuk header, `<td>` untuk data). Jadi tabel ini punya 5 kolom (Judul, Pengarang, Tahun, Stok, Aksi) dan 5 baris data buku.

Kolom "Aksi"

```
<td>
  <button type="button">Edit</button>
  <button type="button">Hapus</button>
</td>
```

- Setiap baris punya 2 tombol: **Edit** dan **Hapus**.
- `type="button"` artinya tombol ini **tombol biasa**, bukan tombol submit form (beda dengan tombol "Simpan" di halaman form — lihat [dokumentasi buku/tambah.html §4.5](#)). Kalau `type` tidak ditulis dan tombol ini berada **di dalam** sebuah `<form>`, browser akan menganggapnya tombol submit secara default — makanya di sini sengaja ditulis eksplisit `type="button"`.
- **Penting untuk dipahami:** tombol ini **belum berfungsi apa-apa**. Belum ada JavaScript yang menghubungkan klik tombol dengan aksi edit/hapus data. Ini baru kerangka tampilan (UI), logikanya menyusul di jobsheet berikutnya saat belajar JavaScript.

3.3 Data yang Ditampilkan (Dummy)

Ada 5 baris data buku yang diketik manual langsung di HTML:

Judul	Pengarang	Tahun	Stok
Laskar Pelangi	Andrea Hirata	2005	4
Bumi Manusia	Pramoedya Ananta Toer	1980	2
Negeri 5 Menara	Ahmad Fuadi	2009	0
Filosofi Teras	Henry Manampiring	2018	5
Ronggeng Dukuh Paruk	Ahmad Tohari	1982	1

Karena ini **HTML statis** (belum terhubung database), data ini tidak bisa ditambah/diubah lewat aplikasi — untuk menambah baris baru, kamu harus mengedit langsung kode HTML-nya (copy-paste satu blok `<tr>...</tr>` lalu ganti isinya).

3.4 Kenapa Pakai `<table>`, Bukan `<div>` Bertumpuk?

Tabel adalah pilihan tag yang tepat di sini karena datanya memang **tabular** — punya kolom dan baris yang jelas relasinya (setiap baris adalah satu buku, setiap kolom adalah satu atribut buku). Browser dan screen reader juga memahami struktur `<table>` secara khusus (misalnya bisa mengumumkan "kolom Tahun, baris 2, isinya 1980"), yang tidak bisa didapat kalau memakai `<div>` biasa.

Lanjut ke: [Penjelasan buku/tambah.html](#)

4. Penjelasan buku/tambah.html (Form Tambah Buku)

File ini menampilkan **form** (formulir isian) untuk menambah data buku baru. Ini adalah file pertama di jobsheet ini yang memperkenalkan elemen `<form>` dan berbagai jenis `<input>`.

4.1 Kode Form Lengkap

```
<form>
  <p>
    <label for="judul">Judul</label><br>
    <input type="text" id="judul" name="judul" required>
  </p>
  <p>
    <label for="pengarang">Pengarang</label><br>
```

```

    <input type="text" id="pengarang" name="pengarang" required>
  </p>
  <p>
    <label for="tahun">Tahun Terbit</label><br>
    <input type="number" id="tahun" name="tahun" min="1900" max="2026" required>
  </p>
  <p>
    <label for="isbn">ISBN</label><br>
    <input type="text" id="isbn" name="isbn">
  </p>
  <p>
    <label for="stok">Stok</label><br>
    <input type="number" id="stok" name="stok" min="0" required>
  </p>
  <p>
    <label for="kategori">Kategori</label><br>
    <select id="kategori" name="kategori">
      <option value="fiksi">Fiksi</option>
      <option value="non-fiksi">Non-Fiksi</option>
      <option value="referensi">Referensi</option>
    </select>
  </p>
  <p>
    <button type="submit">Simpan</button>
  </p>
</form>

```

4.2 Elemen <form>

```

<form>
  ...
</form>

```

<form> membungkus semua elemen isian (input) yang akan dikirim bersama-sama ketika tombol submit ditekan. Perhatikan tag <form> di sini **tidak punya atribut** `action` **maupun** `method` — artinya jika tombol "Simpan" ditekan sekarang, form ini **belum mengirim data ke mana pun** (browser hanya akan reload halaman yang sama). Ini sesuai catatan di README: form-nya "belum diproses". Menghubungkan form ke penyimpanan data (`action="..."`, `method="post"`, lalu backend yang memprosesnya) adalah materi jobsheet selanjutnya.

4.3 Pola Setiap Isian Form: <label> + <input>

Setiap isian dibungkus <p> (paragraf) dan berisi sepasang <label> – <input>, misalnya:

```

<p>
  <label for="judul">Judul</label><br>

```

```
<input type="text" id="judul" name="judul" required>
</p>
```

- `<label for="judul">Judul</label>` — teks keterangan "Judul" yang tampil di sebelah/atas kotak isian. Atribut `for="judul"` **menghubungkan** label ini ke input yang `id`-nya `judul`.
Manfaatnya: kalau pengguna klik tulisan label "Judul", browser otomatis memfokuskan kursor ke kotak input-nya — ini praktik penting untuk aksesibilitas dan kenyamanan pengguna (terutama di HP, area klik jadi lebih besar).
- `<br>` — line break, membuat input pindah ke baris baru di bawah label (bukan sejajar di kanan label).
- `<input>` — kotak isian itu sendiri. Atribut pentingnya:
 - `type="text"` — jenis input berupa teks bebas.
 - `id="judul"` — identitas unik elemen ini di halaman, dipakai untuk dihubungkan dengan `<label for="judul">` di atas.
 - `name="judul"` — nama field yang akan dikirim ke server saat form di-submit (contoh: `judul=Laskar+Pelangi`). `id` **dan** `name` **sengaja dibuat sama nilainya** di form ini untuk memudahkan, tapi keduanya punya fungsi berbeda: `id` untuk dihubungkan ke `<label>` / CSS / JS di **halaman ini**, sedangkan `name` untuk **data yang dikirim** ke server.
 - `required` — atribut validasi bawaan HTML5: browser akan menolak submit form (dan menampilkan pesan peringatan) kalau field ini dikosongkan.

4.4 Jenis-Jenis Input yang Dipakai

Field	Kode	Penjelasan
Judul	<code><input type="text" ... required></code>	Teks bebas, wajib diisi.
Pengarang	<code><input type="text" ... required></code>	Teks bebas, wajib diisi.
Tahun Terbit	<code><input type="number" min="1900" max="2026" required></code>	Hanya menerima angka; <code>min / max</code> membatasi rentang tahun yang masuk akal (1900–2026) langsung dari browser, tanpa perlu JavaScript tambahan.
ISBN	<code><input type="text" ... ></code> (tanpa <code>required</code>)	Teks bebas, boleh dikosongkan — tidak semua buku lama punya ISBN.

Field	Kode	Penjelasan
Stok	<code><input type="number" min="0" required></code>	Angka, minimal 0 (stok tidak mungkin negatif), wajib diisi.
Kategori	<code><select></code> dengan <code><option></code>	Lihat penjelasan di bawah.

`<select>` dan `<option>` – Dropdown Pilihan

```
<select id="kategori" name="kategori">
  <option value="fiksi">Fiksi</option>
  <option value="non-fiksi">Non-Fiksi</option>
  <option value="referensi">Referensi</option>
</select>
```

- `<select>` membuat **menu dropdown** (kotak pilihan yang bisa diklik untuk membuka daftar opsi).
- Setiap `<option>` adalah satu pilihan di dalam dropdown itu.
 - `value="fiksi"` — nilai yang **benar-benar dikirim** ke server saat dipilih.
 - Fiksi (teks di antara tag) — yang **tampil** ke pengguna di layar.
 - Nilai `value` dan teks tampilan **tidak harus sama**; di sini kebetulan mirip (non-fiksi vs Non-Fiksi) supaya konsisten dan mudah dibaca kode maupun tampilannya.
- Berbeda dengan field lain, `<select>` di sini **tidak** dipasang `required` karena selalu ada opsi terpilih secara default (opsi pertama, Fiksi), jadi field ini otomatis "terisi".

4.5 Tombol Submit

```
<button type="submit">Simpan</button>
```

- `type="submit"` membuat tombol ini, saat diklik, **mengirimkan seluruh isi form** (sesuai aturan `action/method` di tag `<form>` — yang saat ini belum diisi). Ini beda dengan tombol `type="button"` di [buku/list.html](#) yang tidak melakukan submit apa pun.
- Karena `<form>` belum punya `action`, saat ini menekan "Simpan" hanya akan me-reload halaman yang sama tanpa efek apa pun yang terlihat.

4.6 Ringkasan Alur Belajar

Urutan penting yang perlu diingat dari file ini:

1. `<form>` = wadah pengiriman data.

2. `<label for="...">` dipasangkan dengan `<input id="...">` yang sama.
3. `name="..."` menentukan nama data yang dikirim ke server.
4. Atribut HTML5 (`required`, `min`, `max`) memberi validasi dasar **tanpa JavaScript**.
5. `<select>` / `<option>` dipakai kalau pilihan pengguna terbatas pada beberapa opsi tetap (bukan teks bebas).

Lanjut ke: [Penjelasan anggota/list.html](#)

5. Penjelasan anggota/list.html (Daftar Anggota)

File ini adalah **tugas mandiri** di jobsheet 1 — polanya sengaja dibuat mirip dengan [buku/list.html](#) supaya kamu belajar menerapkan sendiri konsep tabel HTML pada data yang berbeda (anggota, bukan buku).

5.1 Kode Lengkap

```
<table>
  <thead>
    <tr>
      <th>No. Anggota</th>
      <th>Nama</th>
      <th>Alamat</th>
      <th>No. HP</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>A001</td>
      <td>Siti Aminah</td>
      <td>Malang</td>
      <td>0812xxxx</td>
      <td>
        <button type="button">Edit</button>
        <button type="button">Hapus</button>
      </td>
    </tr>
    <tr>
      <td>A002</td>
      <td>Budi Santoso</td>
      <td>Batu</td>
      <td>0813xxxx</td>
      <td>
        <button type="button">Edit</button>
      </td>
    </tr>
  </tbody>
</table>
```

```

        <button type="button">Hapus</button>
      </td>
    </tr>
  </tbody>
</table>

```

5.2 Apa yang Sama dengan buku/list.html ?

Strukturnya **identik** dengan daftar buku: `<table>` → `<thead>` (baris judul kolom) → `<tbody>` (baris-baris data) → tiap baris `<tr>` berisi sel `<td>`, dan kolom terakhir "Aksi" berisi dua tombol `type="button"` (Edit & Hapus) yang juga **belum berfungsi**. Kalau bagian ini belum jelas, baca dulu penjelasan detailnya di [dokumentasi buku/list.html §3.2](#).

5.3 Apa yang Berbeda?

Perbedaannya hanya pada **nama kolom** dan **datanya**, karena entitas yang ditampilkan beda (anggota, bukan buku):

Kolom Buku	Kolom Anggota
Judul	No. Anggota
Pengarang	Nama
Tahun	Alamat
Stok	No. HP
Aksi	Aksi

Data yang ditampilkan (2 baris, lebih sedikit dari daftar buku yang 5 baris):

No. Anggota	Nama	Alamat	No. HP
A001	Siti Aminah	Malang	0812xxxx
A002	Budi Santoso	Batu	0813xxxx

Nomor No. HP sengaja ditulis 0812xxxx / 0813xxxx (bukan angka lengkap asli) karena ini hanya **data contoh** untuk latihan, bukan data pribadi sungguhan — praktik yang baik saat membuat contoh/dummy data.

5.4 Perhatikan: Menu Navigasi

```
<nav>
  <ul>
    <li><a href="../index.html">Beranda</a></li>
    <li><a href="../buku/list.html">Daftar Buku</a></li>
    <li><a href="list.html">Daftar Anggota</a></li>
    <li><a href="tambah.html">Tambah Anggota</a></li>
  </ul>
</nav>
```

Perhatikan bahwa file ini memuat 4 tautan navigasi, termasuk "**Tambah Anggota**" menuju `tambah.html` — berbeda dari menu di `index.html` dan `buku/*.html` yang tidak menyertakan tautan tersebut. Ini bagian dari tugas mandiri: pastikan menu di setiap halaman **konsisten** menampilkan seluruh 5 halaman jobsheet ini (Beranda, Daftar Buku, Tambah Buku, Daftar Anggota, Tambah Anggota) — sebagai latihan, coba periksa dan lengkapi sendiri menu di `index.html`, `buku/list.html`, dan `buku/tambah.html` supaya tautan "Daftar Anggota" **dan** "Tambah Anggota" sama-sama muncul di semua halaman.

Lanjut ke: [Penjelasan anggota/tambah.html](#)

6. Penjelasan anggota/tambah.html (Form Tambah Anggota)

Sama seperti `anggota/list.html`, file ini juga bagian **tugas mandiri**: polanya dibuat mirip [buku/tambah.html](#), tapi untuk data anggota.

6.1 Kode Lengkap

```
<form>
  <p>
    <label for="nama">Nama</label><br>
    <input type="text" id="nama" name="nama" required>
  </p>
  <p>
    <label for="no_anggota">No. Anggota</label><br>
    <input type="text" id="no_anggota" name="no_anggota" required>
  </p>
  <p>
    <label for="alamat">Alamat</label><br>
    <input type="text" id="alamat" name="alamat">
  </p>
</form>
```

```

<label for="no_hp">No. HP</label><br>
<input type="text" id="no_hp" name="no_hp">
</p>
<p>
<button type="submit">Simpan</button>
</p>
</form>

```

6.2 Apa yang Sama dengan buku/tambah.html ?

Pola dasarnya identik dengan form tambah buku: setiap field dibungkus `<p>`, berisi pasangan `<label for="..."> + <input id="..." name="...">`, dan diakhiri tombol `<button type="submit">Simpan</button>`. Jika belum paham cara kerja pasangan label-input atau kenapa tombolnya `type="submit"`, baca dulu [dokumentasi buku/tambah.html §4.3–4.5](#).

6.3 Apa yang Berbeda?

Field yang diminta lebih sedikit dan lebih sederhana dibanding form buku — **semua** field di sini bertipe `type="text"` (tidak ada `type="number"` atau `<select>`, karena data anggota di jobsheet ini tidak butuh angka atau pilihan kategori):

Field	Kode Input	Wajib Diisi?
Nama	<code><input type="text" id="nama" name="nama" required></code>	Ya (required)
No. Anggota	<code><input type="text" id="no_anggota" name="no_anggota" required></code>	Ya (required)
Alamat	<code><input type="text" id="alamat" name="alamat"></code>	Tidak
No. HP	<code><input type="text" id="no_hp" name="no_hp"></code>	Tidak

Perhatikan penamaan atribut `id / name` yang memakai **underscore** (`no_anggota`, `no_hp`) karena nama field terdiri dari beberapa kata. Ini konvensi penamaan yang umum di HTML/pemrograman supaya nama variabel tidak mengandung spasi (spasi pada `name` bisa menimbulkan masalah saat data dikirim/diproses).

6.4 Kenapa "No. Anggota" Berupa Teks, Bukan Angka?

Menarik untuk diperhatikan: field **"No. Anggota"** memakai `type="text"`, bukan `type="number"`, meskipun namanya mengandung kata "nomor". Ini karena format nomor anggota di aplikasi

ini adalah kombinasi huruf + angka, contohnya A001, A002 (lihat data di [anggota/list.html](#)) — dan `type="number"` hanya menerima digit murni, tidak bisa memuat huruf A. Ini contoh nyata pentingnya memilih `type` input sesuai **bentuk data sesungguhnya**, bukan sekadar namanya.

6.5 Latihan Reflektif

Sebagai latihan mandiri, coba bandingkan sendiri form ini dengan form buku dan jawab pertanyaan berikut untuk menguji pemahaman:

1. Kenapa field "Alamat" dan "No. HP" tidak diberi `required`, sedangkan "Nama" dan "No. Anggota" diberi?
2. Apa yang akan terjadi (di browser) kalau kamu klik tombol "Simpan" tanpa mengisi field "Nama"? Coba buka filenya di browser dan praktikkan.
3. Form ini juga **belum punya** `action` pada tag `<form>`-nya — apa dampaknya saat tombol "Simpan" ditekan?

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

7. Rangkuman & Latihan Lanjutan

7.1 Rangkuman Keseluruhan Jobsheet 1

File	Fungsi	Elemen Baru yang Dipelajari
index.html	Beranda + ringkasan statistik	<code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>footer</code>
buku/list.html	Tabel daftar buku	<code>table</code> , <code>thead</code> , <code>tbody</code> , <code>tr</code> , <code>th</code> , <code>td</code> , <code>button type="button"</code>
buku/tambah.html	Form tambah buku	<code>form</code> , <code>label</code> , <code>input (text / number)</code> , <code>select / option</code> , <code>button type="submit"</code> , atribut <code>required / min / max</code>
anggota/list.html	Tabel daftar anggota (tugas mandiri)	Penerapan ulang pola tabel pada data berbeda

File	Fungsi	Elemen Baru yang Dipelajari
anggota/tambah.html	Form tambah anggota (tugas mandiri)	Penerapan ulang pola form pada data berbeda

Secara umum, jobsheet 1 mengajarkan **3 pola HTML** yang akan terus dipakai berulang-ulang di seluruh aplikasi:

1. **Kerangka halaman** (`header + nav + main + footer`) — sama di semua 5 halaman.
2. **Tabel data** (`table / thead / tbody / tr / th / td`) — dipakai di kedua halaman "list".
3. **Form isian** (`form / label / input / select / button`) — dipakai di kedua halaman "tambah".

7.2 Hal yang Perlu Diingat: Ini Baru Kerangka

Beberapa hal yang **sengaja belum ada** di jobsheet ini (bukan bug, memang belum waktunya dipelajari):

- **Tidak ada CSS** → tampilan masih polos, tanpa warna/tata letak rapi.
- **Tidak ada JavaScript** → tombol Edit/Hapus di tabel belum berfungsi.
- **Form belum terhubung ke mana pun** (`<form>` tanpa `action`) → tombol "Simpan" belum benar-benar menyimpan data.
- **Data tabel adalah data statis (dummy)**, diketik manual di HTML, bukan diambil dari database.

Semua ini akan mulai dipelajari di jobsheet berikutnya.

7.3 Cara Mencoba Sendiri

1. Buka file `index.html` langsung di browser (klik dua kali, atau klik kanan → *Open with* → pilih browser). Belum butuh server apa pun.
2. Klik menu navigasi di header untuk berpindah antar halaman, perhatikan bagaimana `href` pada tiap `<a>` menentukan halaman tujuan.
3. Coba isi form di `buku/tambah.html` atau `anggota/tambah.html`, lalu klik "Simpan" — amati bahwa tidak terjadi apa-apa selain reload halaman (karena belum ada `action`).
4. Coba kosongkan salah satu field yang punya atribut `required`, lalu klik "Simpan" — perhatikan browser menampilkan peringatan validasi otomatis bawaan HTML5.

7.4 Ide Latihan Tambahan (Opsional)

Untuk memperdalam pemahaman, coba lakukan sendiri (tidak wajib, tapi sangat disarankan untuk latihan):

1. **Lengkapi konsistensi menu** — tambahkan tautan "Daftar Anggota" dan "Tambah Anggota" ke menu `<nav>` di `index.html`, `buku/list.html`, dan `buku/tambah.html` (lihat catatan di [dokumentasi anggota/list.html §5.4](#)).
2. **Tambah 2 baris data buku baru** di `buku/list.html` dengan meng-copy satu blok `<tr>...</tr>` lalu mengganti isinya.
3. **Tambah kolom baru** di tabel anggota, misalnya "Tanggal Bergabung", lengkap dengan `<th>` dan `<td>`-nya di setiap baris.
4. **Tambah field baru** di form tambah anggota, misalnya "Email" memakai `<input type="email">` (`type="email"` otomatis memvalidasi format alamat email tanpa perlu JavaScript tambahan).

Selamat belajar — kalau ada bagian yang masih membingungkan, coba baca ulang [konsep dasar di file 1](#), karena hampir semua istilah teknis di file-file lain dijelaskan di sana.